



ok

Below is the final upgraded **corporate-level** CLAUDE.md for Yewogen Derash, now including the approved tools stack, build order, security rules, campaign-separated querycodes, admin-only financial control, quick-donate flow, verification flow, and corporate UX requirements. You can paste this directly into your repo as CLAUDE.md.

```
# Yewogen Derash – Corporate-Level CLAUDE.md

## 1. Project Mission

Build **Yewogen Derash**, a secure, corporate-grade crowdfunding platform for Ethiopia.

The platform must:

- Support Ethiopian payment methods and international payment methods through a single interface.
- Give every campaign its own dedicated querycode and QR code so donations never mix.
- Provide a full campaign page plus a fast-donate flow for QR scans.
- Require strict campaign owner onboarding with ID upload, face capture, terms acceptance.
- Provide an admin-only corporate control panel with financial ledgers, KYC review, and reporting.
- Send campaign owners WhatsApp alerts after each successful donation via CallMeBot or similar.
- Avoid scam-like UX by showing verification, public trust signals, and clear payout timelines.

Claude Code must treat this document as the source of truth.

---

## 2. Approved Tools and Stack

Use only the tools below unless a strong technical reason is documented.

### 2.1 Frontend
- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui or Radix UI
- React Hook Form
- Zod

### 2.2 Backend
- Node.js
- Express or NestJS
- Prisma
- PostgreSQL

### 2.3 Authentication
- Auth.js / NextAuth or a clean custom JWT-based auth system
```

- Role-based access control

2.4 File Handling

- Secure upload handling for ID documents and campaign media
- S3-compatible storage or Cloudinary
- File type validation and size limits

2.5 Payments

- Chapa payment gateway integration.[web:57][web:58][web:59]
- Webhook verification must be implemented.

2.6 QR / Querycode

- QR code generation library
- Unique querycode generator per campaign

2.7 Notifications

- CallMeBot WhatsApp notifications

2.8 Analytics

- Recharts or Chart.js

2.9 Testing

- Vitest or Jest
- Playwright for end-to-end tests

2.10 Deployment

- Vercel for frontend
- Secure backend host for API and database
- Environment variables only for secrets

Do not introduce random tools without updating this document.

3. Core Platform Rules

3.1 Trust First

The platform must feel safe, transparent, and auditable.

- Every campaign must have a verified owner profile.
- Every campaign must have a unique querycode and QR code.
- Every donation must be tied to exactly one campaign.
- Every payout must be recorded in a separate payout ledger.
- Every admin action must be logged.

3.2 No Fund Mixing

Funds for different campaigns must never be mixed.

- Each campaign gets its own donation ledger.
- Each campaign gets its own payout summary.
- Campaign owners may only see their own campaigns and balances.
- Only admin can see the full system-wide financial view.

3.3 Corporate UX

The site must look polished, minimal, and trustworthy.

- Clean typography.
- Strong visual hierarchy.
- Consistent spacing.
- Professional color palette.
- No clutter.
- No exaggerated claims.
- No scam-like urgency tricks.

4. Build Order

Claude Code must build the project in this order:

1. Scaffold project structure.
2. Build authentication and roles.
3. Build database schema and Prisma models.
4. Build public website pages.
5. Build owner onboarding and verification flow.
6. Build campaign creation and admin approval flow.
7. Build unique querycode and QR generation.
8. Build standard donation flow.
9. Build quick-donate QR flow.
10. Integrate payment gateway and webhook handling.
11. Add WhatsApp alerts.
12. Build admin panel with ledgers, reviews, CMS, and analytics.
13. Add legal pages, settings, and content management.
14. Harden security, logs, and audit trails.
15. Add tests and deployment docs.

Claude should not jump ahead without finishing the earlier steps.

5. Public Website Structure

The public site must have five main pages with subpages and clear purposes.

5.1 Home

****Route:**** `/`

Purpose:

- Introduce Yewogen Derash.
- Explain trust, verification, and payment support.
- Push users either to browse campaigns or start a campaign.

Sections:

- Hero banner.
- Featured campaigns.
- How it works.
- Trust and verification.
- Success stories.
- Footer with legal and support links.

5.2 Campaigns

****Route:**** `/campaigns`

Purpose:

- Show all public campaigns.
- Let donors browse by category, location, status, and progress.

Subpages:

- `/campaigns/[campaignSlug]` – full campaign page.
- `/campaigns/[campaignSlug]/updates` – public updates.
- `/campaigns/[campaignSlug]/donate` – standard donation page.

Campaign page must show:

- Title.
- Hero image.
- Story.
- Progress bar.
- Target and raised amount.
- Verified owner badge.
- Verification status.
- Campaign-specific querycode badge.
- Donate button.

5.3 Quick Donate

****Route:**** `/q/[queryCode]`

Purpose:

- Fast donation flow for people who scan QR or type the code.
- Minimal steps, mobile-first, low friction.

Behavior:

- Lands directly on the exact campaign donation page.
- Shows campaign title, image, verification badge, target, and progress.
- Shows preset donation amounts.
- Shows payment method selection.
- Shows a simple confirm and pay flow.
- After donation success, shows thank-you screen and then resets or exits cleanly.

This page must be separate per campaign, not shared.

5.4 Start a Campaign

****Route:**** `/start`

Purpose:

- Register campaign owners.
- Collect documents.
- Perform face capture.
- Accept legal terms and hosting fees.
- Submit for review.

Subpages:

- `/start/register`
- `/start/verify`
- `/start/terms`
- `/start/documents`
- `/start/review`

5.5 Support and Legal

****Route:**** `/support`

Purpose:

- Provide fees, legal text, help, FAQ, privacy, and contact.

Subpages:

- `/fees`
- `/terms`
- `/privacy`
- `/faq`
- `/contact`

6. Campaign Owner Onboarding

A campaign owner must go through a strict procedure before being allowed to create or

6.1 Required Steps

1. Create account with email and phone.
2. Verify email and phone with OTP.
3. Accept terms and conditions.
4. Accept hosting fee and platform fee policy.
5. Upload identity documents.
6. Capture live face for verification.
7. Submit proof documents based on campaign type.
8. Wait for admin review.
9. Receive approval and unique owner verification ID.
10. Create campaign(s).

6.2 Required Documents

Minimum required documents:

- National ID or passport.
- Selfie / face capture with liveness check.
- Phone number.
- Email address.
- Payout account details.
- Proof of address if needed.
- Supporting document for the campaign purpose.

Campaign-type examples:

- Medical campaign: hospital letter, doctor letter, or treatment estimate.
- Education campaign: school letter, admission letter, or fee statement.
- Community campaign: NGO letter, local authority letter, or project proof.
- Business campaign: business registration, tax or permit documents, cost estimate.

6.3 Document Upload Rules

- Frontend must support clear file upload with preview.
- File types: image, PDF.
- Maximum size limits.
- Upload progress and success message.

- If image quality is poor, request resubmission.
- All uploaded documents must be stored securely and accessible only to authorized ac

7. Verification and Trust Model

7.1 Biometric Verification

The system must support face capture for campaign owner identity verification.

- Capture live face.
- Perform liveness detection.
- Compare with uploaded ID and account identity.
- Mark status as pending, verified, rejected, or resubmit.

7.2 Campaign Author ID

Once verified, each campaign owner receives a public author profile with:

- Full name.
- Photo.
- Verification badge.
- Unique verification code.
- QR code.
- Mulesoo stamp.
- List of campaigns owned.
- Verification status and date.

7.3 Mulesoo Stamp

The Mulesoo stamp is a platform trust seal.

- It should appear only after admin approval.
- It must be visible on owner profile and verified campaigns.
- It must not be editable by the user.

8. Campaign and Querycode Rules

8.1 One Campaign, One Querycode

- Every campaign must have its own unique querycode.
- Every campaign must have its own QR code.
- A QR code must open only that campaign's quick-donate page.
- One querycode must never point to more than one campaign.

8.2 One Campaign, One Ledger

- Each campaign must have a separate donation ledger.
- Each campaign must have a separate payout summary.
- Campaign owners must not see other owners' campaigns or funds.
- Admin may see all ledgers.

8.3 Donation Routing

When a donor scans the QR:

- The system identifies the exact campaign.
- The donor lands on the campaign's quick-donate page.
- The donor sees campaign details and verified status.
- Payment is processed only for that campaign.

9. Donation Flow

9.1 Standard Donation Flow

1. User opens full campaign page.
2. User clicks donate.
3. User enters amount.
4. User selects payment method.
5. Backend creates a pending donation record.
6. Backend creates payment session with gateway.
7. User completes payment.
8. Gateway sends webhook.
9. Backend verifies webhook.
10. Donation status is updated.
11. Campaign total is updated.
12. Campaign owner receives WhatsApp alert.
13. Donor sees thank-you receipt.

9.2 Quick Donate Flow

1. User scans QR or enters querycode.
2. User lands on `/q/[queryCode]`.
3. User sees short campaign summary and payment box.
4. User chooses amount.
5. User pays.
6. Backend verifies payment.
7. Owner gets alert.
8. Thank-you screen appears.
9. Session closes or resets.

9.3 Donation State Machine

Donation status values must include:

- `PENDING`
- `SUCCESS`
- `FAILED`
- `CANCELLED`
- `REFUNDED`
- `DISPUTED`

10. Payment Integration

Use Chapa as the payment gateway integration layer.[web:57][web:58][web:59]

10.1 Payment Requirements

- Support Ethiopian local payment methods.
- Support international methods where the gateway allows.
- Create payment session.
- Handle return URL.
- Handle webhook confirmation.
- Verify payment before marking success.

10.2 Webhook Handling

- Webhooks must be verified before acceptance.[web:57][web:59]
- Match payment to the correct campaign and donation record.
- Ignore unknown or duplicate webhook events.
- Log every webhook event.

10.3 Receipt and Reference

Each donation receipt must include:

- Donation ID.
- Campaign name.
- Amount.
- Currency.
- Timestamp.
- Transaction reference.
- Payment status.

11. WhatsApp Alerts

After each successful donation:

- Send a WhatsApp alert to the campaign owner.
- Include campaign name, amount, donor reference if allowed, and time.
- Use CallMeBot or a similar WhatsApp notification service.
- Log whether notification succeeded or failed.

Owners can enable or disable alerts in their dashboard settings.

12. Admin Panel Requirements

The admin panel must be corporate-level and admin-only.

12.1 Access Control

- Only admin can log in.
- Campaign owners must not access the financial control room.
- Donor users must not access admin pages.
- Use role-based access control.

12.2 Admin Dashboard

Show:

- Total campaigns.
- Total verified owners.
- Total donations.
- Total raised amount.
- Total payouts.
- Pending reviews.
- Failed verifications.
- Fraud flags.

12.3 Campaign Management

Admin can:

- Approve.
- Reject.
- Suspend.
- Feature.
- Archive.

Admin sees:

- Campaign details.
- Owner details.
- Documents.
- Verification status.
- Querycode.
- Payment summary.

12.4 Financial Control

Admin sees:

- Donation ledger by campaign.
- Payment gateway status.
- Refunds.
- Chargebacks.
- Payout eligibility.
- Payout execution.
- Audit logs.

12.5 KYC and Compliance

Admin sees:

- Document uploads.
- Face verification status.
- Review queue.
- Verification history.
- Notes and decisions.

12.6 CMS and Content

Admin can manage:

- Home page content.
- Featured campaigns.
- About page.
- FAQ.
- Terms.
- Privacy.
- Fees.
- Contact details.
- Banners and announcements.

12.7 Audit Logs

Log every admin action:

- Login.
- Approval.
- Rejection.
- Payout.
- Content update.
- User ban.
- Fee change.

12.8 Analytics

Show charts for:

- Donations over time.
- Campaign performance.
- Category performance.
- Owner verification rate.
- Refunds and failures.
- Payout timelines.

13. Anti-Fraud and Risk Controls

The system must actively prevent misuse.

13.1 Required Controls

- Email and phone verification.
- Identity document upload.
- Face capture and liveness check.
- Manual admin review before publishing.
- Rate limits on campaign creation.
- Spam and bot protection.
- Fraud alert flags for suspicious behavior.
- Manual payout approval for new owners.

13.2 Safe Money Handling

- Do not allow campaign owners to self-release funds without approval.
- Do not allow unverified users to create live campaigns.
- Do not allow undeclared campaign duplicates.

- Do not allow shared querycodes across campaigns.

13.3 Public Trust Signals

Show on public pages:

- Verified badge.
- Last reviewed date.
- Owner verification code.
- Campaign status.
- Funding progress.
- Report abuse button.

14. Data Model Requirements

14.1 Users

Fields:

- id
- name
- email
- phone
- passwordHash
- role
- verificationStatus
- createdAt
- updatedAt

14.2 Campaign Owners

Fields:

- id
- userId
- idNumber
- biometricStatus
- authorCode
- mulesooVerified
- payoutAccount
- createdAt
- updatedAt

14.3 Campaigns

Fields:

- id
- ownerId
- title
- slug
- description
- category
- targetAmount
- currentAmount
- currency
- status
- queryCode
- qrCodeUrl
- createdAt

- updatedAt

14.4 Donations

Fields:

- id
- campaignId
- donorId optional
- amount
- currency
- paymentMethod
- gatewayTransactionId
- status
- webhookPayload
- createdAt
- updatedAt

14.5 Payouts

Fields:

- id
- campaignId
- ownerId
- amount
- status
- payoutReference
- approvedByAdminId
- createdAt
- updatedAt

14.6 Verification Documents

Fields:

- id
- ownerId
- documentType
- fileUrl
- status
- adminNote
- createdAt
- updatedAt

14.7 Notifications

Fields:

- id
- ownerId
- campaignId
- channel
- message
- status
- createdAt
- updatedAt

15. UX Rules

15.1 Public Pages

- Should feel trustworthy and corporate.

- Should not be crowded.
- Should not use aggressive scam-like language.
- Should clearly show verified status and campaign ownership.

15.2 Quick Donate

- Must be simple.
- Must work on mobile.
- Must require minimal taps.
- Must never hide which campaign is receiving the money.

15.3 Owner Dashboard

- Show only the owner's data.
- Include campaign performance.
- Include notifications.
- Include payout status.
- Include querycode download and QR export.

15.4 Admin Panel

- Dense, professional, and data-oriented.
- Tables, filters, charts, review queues, and logs.
- No public-style marketing layout.

16. Legal and Compliance Expectations

Claude must implement the structure for legal compliance, even if the final legal text

The platform must include:

- Terms and conditions.
- Privacy policy.
- Hosting fee disclosure.
- Payout policy.
- Refund policy.
- Abuse and reporting policy.
- Verification and document consent policy.
- Biometric data consent policy.

The system must require acceptance of terms before owner onboarding continues.

17. Implementation Rules

- Never let campaign funds mix.
- Never allow unverified campaign owners to publish instantly.
- Never allow owners to view other owners' financial data.
- Never allow a QR code to point to more than one campaign.
- Never mark a payment successful without webhook verification.
- Never expose admin endpoints publicly.
- Never store secrets in code.
- Never bypass audit logs.
- Never add tools without checking this document first.

18. Definition of Done

A feature is done only when it:

- Works locally.
- Has validation.
- Has error handling.
- Has permissions enforced.
- Has logs or audit records if relevant.
- Has test coverage or is testable.
- Has documentation or usage notes.

19. Final Instruction

Build this as a serious, trustworthy, corporate crowdfunding platform for Ethiopia.

The experience should feel like:

- a legitimate fintech product,
- a verified donation platform,
- a clean public website,
- and a secure admin-controlled financial system.

Prioritize trust, clarity, separation of funds, and auditability above everything else.